CPSC 481
Artificial Intelligence

Chuck Chekuri

cchekuri@fullerton.edu

LinkedIn

Group

Syllabus

Week 10: Neural Networks as Function Approximators

From Logic and Trees to Continuous Circuits.

Textbook: AIMA 4th Ed. Chapter 21 (Deep Learning).

Core Goal: Understanding how "layers" approximate reality.

The transition from Feature Engineering to Representation Learning.

The Limits of Chapter 19

- Linear/Logistic regression struggle with high-dimensional data (Images).
- Decision Trees become unmanageably deep for pixel-level tasks.
- Deep Learning handles complex algebraic circuits with tunable strengths.
- "Deep" = many layers between input and output.

Deep Neural Network

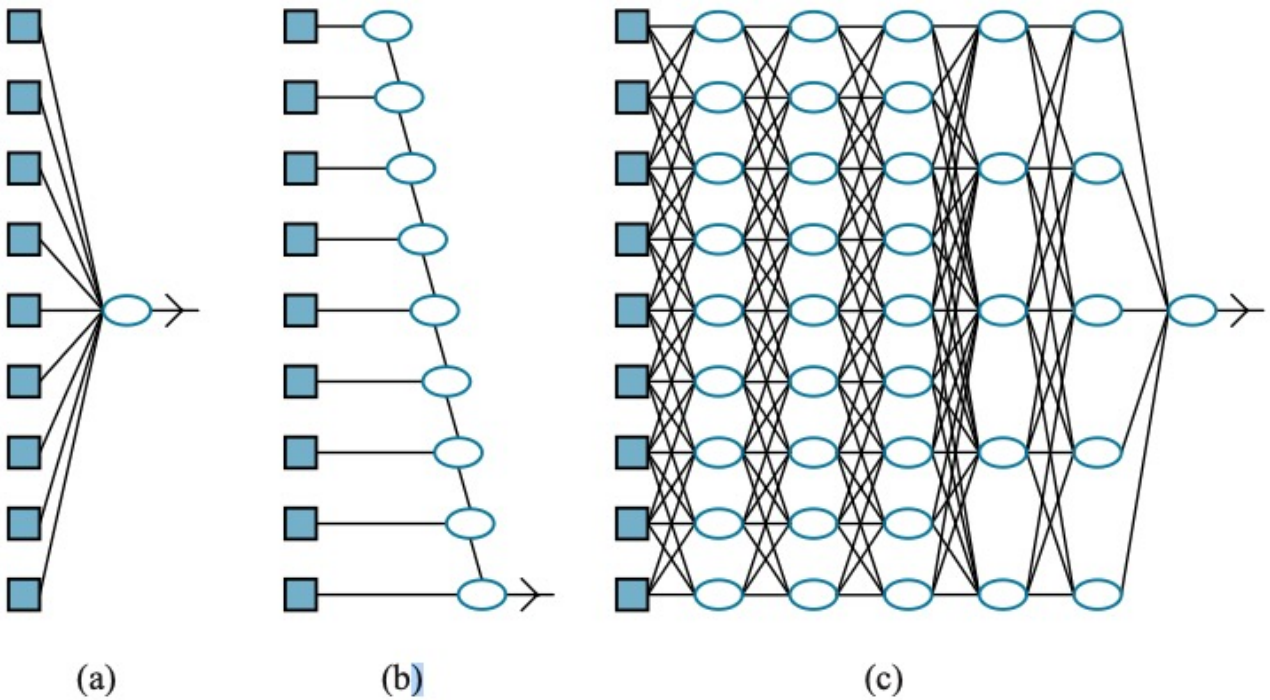

Figure 21.1 (a) A shallow model, such as linear regression, has short computation paths between inputs and output. (b) A decision list network (page 674) has some long paths for some possible input values, but most paths are short. (c) A deep learning network has longer computation paths, allowing each variable to interact with all the others.

Activation Functions

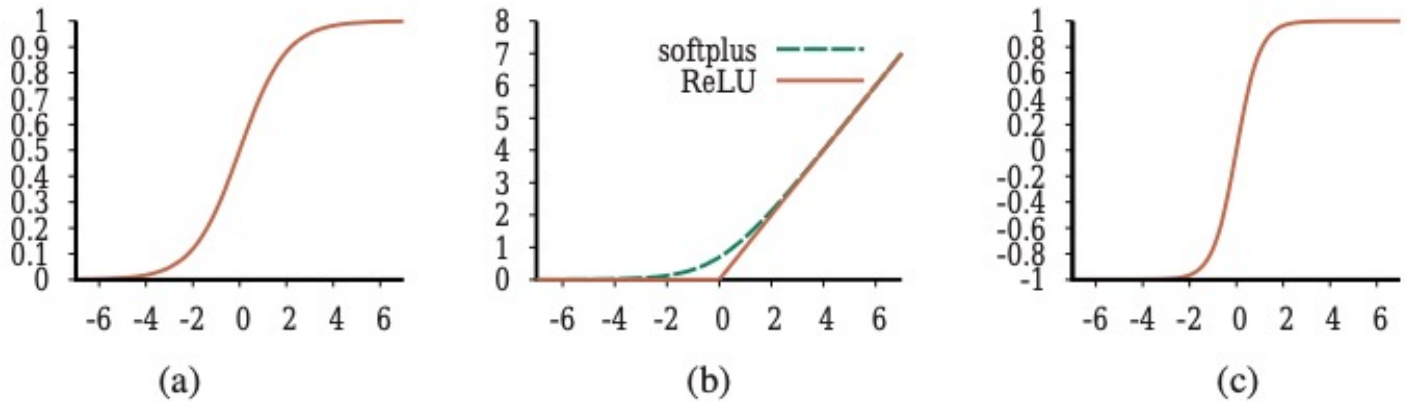

Figure 21.2 Activation functions commonly used in deep learning systems: (a) the logistic or sigmoid function; (b) the ReLU function and the softplus function; (c) the tanh function.

Simple Feed Forward Network

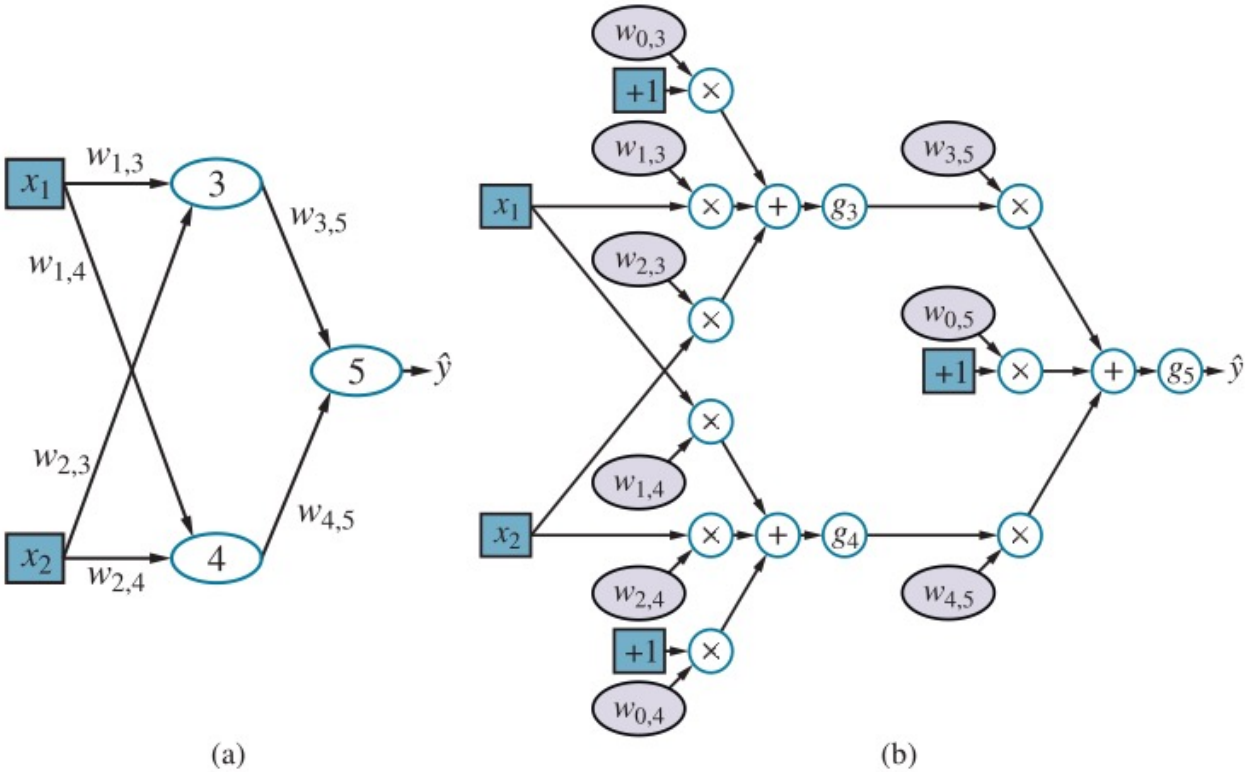

Figure 21.3 (a) A neural network with two inputs, one hidden layer of two units, and one output unit. Not shown are the dummy inputs and their associated weights. (b) The network in (a) unpacked into its full computation graph.

Neural Networks: Biological Roots

- Early work: McCulloch and Pitts (1943).
- Modeled after the "wetware" of the biological brain.
- Resemblance to real neurons is now superficial.
- Focus is now on computational and statistical efficiency.

The Simple Unit: The Perceptron

- Computation: $z = \sum (w_i x_i) + b$.
- Activation: $a = g(z)$.
- Weights (w) and Bias (b) are the "tunable" parameters.
- A single unit is a linear classifier.

Activation Functions: Why Non-Linearity?

- Without non-linearity, multiple layers = one single linear layer.
- Sigmoid: Squashes to [0,1].
- ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$.
- Tanh: Squashes to [-1,1].

The Power of ReLU

- Most popular activation for deep networks.
- Solves the "Vanishing Gradient" problem.
- Computationally cheap: Just a threshold at zero.
- Allows for faster convergence in large-scale models.

Network Structures: Feedforward

- Information flows in one direction: Input -> Hidden -> Output.
- No internal loops (Directed Acyclic Graph).
- Organized in "Layers": Each layer computes a new representation.
- Depth = Number of hidden layers.

The Universal Approximation Theorem

- A single hidden layer can approximate *any* continuous function.
- Why go deep then? Efficiency.
- Deep networks require exponentially fewer units for complex tasks.
- Depth allows for hierarchical feature extraction.

Loss Functions for Neural Nets

- Regression: Mean Squared Error (MSE).
- Classification: Cross-Entropy Loss.
- Loss measures the "distance" between prediction and truth.
- Goal: Minimize Loss across the entire training set.

Gradient Descent Revisited

- Weights are updated: $w \leftarrow w - \eta \frac{\partial L}{\partial w}$.
- η is the learning rate.
- We need the derivative (gradient) of the loss with respect to every weight.
- How do we find this in a 100-layer network?

Backpropagation: The Chain Rule

- The fundamental algorithm of Deep Learning.
- Pass 1 (Forward): Compute outputs and loss.
- Pass 2 (Backward): Compute gradients from output back to input.
- Uses the Calculus Chain Rule to distribute "blame."

Stochastic Gradient Descent (SGD)

- Updating weights after *every* example (Too noisy).
- Batch Gradient Descent: Use the whole dataset (Too slow).
- Mini-batch SGD: Use a small sample (e.g., 32 or 64 examples).
- Best balance of speed and stability.

The Problem of Local Minima

- Loss surfaces in Deep Learning are non-convex (bumpy).
- High-dimensional spaces actually have more "Saddle Points" than minima.
- Momentum: Helps the optimizer roll through small bumps.
- Modern optimizers: Adam, RMSProp.

Overfitting in Neural Networks

- Neural nets have millions of parameters.
- They can easily "memorize" the noise in training data.
- Symptom: Training loss goes to zero, Test loss goes up.
- We need **Regularization**.

Regularization: Dropout

- Randomly "kill" neurons during training (e.g., 50%).
- Forces the network to learn redundant, robust features.
- Prevents any single neuron from becoming "too important."
- Turned off during testing/inference.

Regularization: Weight Decay (L2)

- Add a penalty to the loss for having large weights.
- Loss = Error + $\lambda \sum (w^2)$.
- Keeps the weights small and the model "smooth."
- Forces the network to favor simpler explanations.

Representation Learning

- Old AI: We hand-engineer features (e.g., "detect edges").
- Deep Learning: The network learns which features are important.
- Early layers: Detect edges/colors.
- Deep layers: Detect complex parts/objects.

Linear Algebra: The Real Language

- In practice, we don't code individual neurons.
- We use Matrix Multiplication: $Y = XW + b$.
- GPUs (Graphics Processing Units) are built for this math.
- This is why Deep Learning took off in the 2010s.

Vanishing and Exploding Gradients

- Gradients are multiplied layer-by-layer.
- If weights < 1: Gradient shrinks to zero (Vanishes).
- If weights > 1: Gradient blows up to infinity (Explodes).
- Solutions: Batch Normalization, He Initialization.

Batch Normalization

- Normalize the activations of each layer to have zero mean and unit variance.
- Reduces internal "Covariate Shift."
- Allows for much higher learning rates.
- Acts as a mild form of regularization.

Data Augmentation

- If you don't have enough data, "fake" it.
- For images: Flip, rotate, crop, change color.
- The label stays the same; the input looks new.
- Significant boost in generalization for visual tasks.

Softmax Output Layer

- For multi-class classification (e.g., Digit 0-9).
- Squashes final layer into a probability distribution.
- All outputs sum to 1.0.
- Used with Cross-Entropy loss for training.

Tuning Hyperparameters: The Grid Search

- Learning rate, Batch size, Number of layers, Hidden units.
- Deep learning is sensitive to these choices.
- Often requires many GPU hours to find the "best" set.
- "Neural Architecture Search" (AutoML) automates this.

Convolutional Neural Networks (CNN) Preview

- Specialized for grid-like data (Images).
- Uses "Filters" to share weights across the input.
- Drastically reduces the number of parameters.
- Next week: The specifics of Vision.

Recurrent Neural Networks (RNN) Preview

- Specialized for sequential data (Text, Speech).
- Information persists over time steps.
- Foundation of early machine translation.
- Next week: Dealing with time.

The "Black Box" Problem

- We know the math, but we don't know the "Logic."
- Why did the car steer left? Hard to explain with 50M weights.
- Saliency Maps: Visualizing what the net is "looking at."
- Essential for High-Stakes AI (Medicine, Law).

Hardware for Deep Learning

- CPUs: Good for general logic.
- GPUs: Parallel processing for matrix math.
- TPUs (Tensor Processing Units): Google's specialized AI chips.
- Deep Learning is "Compute Bound."

Embedding Layers

- Turning discrete tokens (Words) into continuous vectors.
- "King - Man + Woman = Queen."
- Captures semantic relationships in a low-dimensional space.
- Foundation of modern NLP.

Autoencoders

- Unsupervised learning with Neural Nets.
- Goal: Compress input to a "Bottleneck" and reconstruct it.
- Learns the most efficient representation of the data.
- Used for denoising and anomaly detection.

Transfer Learning

- Don't train from scratch.
- Use a model trained on a huge dataset (e.g., ImageNet).
- Fine-tune the last few layers for your specific task.
- Saves time, money, and data.

Case Study: AlexNet (2012)

- The "Big Bang" of Deep Learning.
- Won the ImageNet competition by a massive margin.
- 8 layers, 60M parameters.
- Proved that GPUs + Data + Deep Nets = SOTA.

The Vanishing Gradient: ResNets

- How do we train 100+ layers?
- Skip Connections: $y = f(x) + x$.
- Allows gradients to flow directly to earlier layers.
- Winner of ImageNet 2015.

Optimization: Adam

- "Adaptive Moment Estimation."
- Combines Momentum and RMSProp.
- Automatically adjusts learning rate for each weight.
- Most popular optimizer in modern research.

Implementation: PyTorch vs. TensorFlow

- PyTorch: Dynamic graphs, pythonic, researcher favorite.
- TensorFlow/Keras: Static graphs, industry/production standard.
- Both use "Auto-Diff" to handle backprop automatically.
- You only write the Forward pass.

Week 10 Summary

- Neural Nets are universal function approximators.
- Backpropagation + SGD = The learning mechanism.
- Overfitting is solved via Regularization (Dropout/L2).
- Deep Learning = Hierarchical Representation Learning.

Homework: Week 10

- 1. Manual Backprop: Trace the gradients through a 2-node net.
- 2. Reading: Ch. 21.3-21.4 (CNNs and RNNs).
- 3. Lab: Implement a 2-layer MLP in PyTorch for MNIST.

The "Wetware" Philosophy

- "Understanding isn't just a software patch; it requires causal powers."
- Simulated consciousness vs. Biological reality.
- Limits of Strong AI (AIMA Ch. 28).
- Are we just very expensive lookup tables?

Ethics: Energy and Environment

- Training large models consumes massive electricity.
- The "Carbon Footprint" of AI.
- Efficiency is an ethical requirement.
- Small Language Models (SLMs) vs. LLMs.

Review: From Search to Deep Learning

- Search (5-6): Strategic Pathfinding.
- Probability (7-8): Handling Uncertainty.
- Learning (9-10): Discovering Functions.
- We are building a "Complete Agent."

Conclusion & Dismissal

- Deep Learning isn't magic; it's calculus on a massive scale.